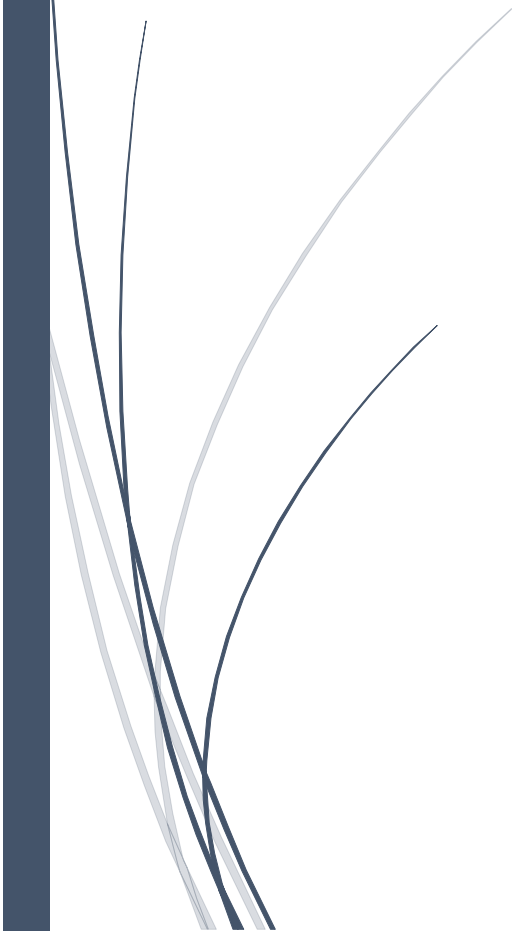
@VANZA

UT2. Sintaxis Básica

CFGS

Desarrollo de Aplicaciones Multiplataforma



Dpto. Informática
IES AUGUSTÓBRIGA (NAVALMORAL DE LA MATA)

Contenido

1. Introducción a los tipos de datos 3
2. Literales, variables y datos en Python. 4
3. Tipo Booleano. 7
3.1. Operaciones con booleanos. 8
3.2. Cortocircuito lógico. 8
4. Comparaciones. 9
5. Tipos numéricos 9
5.1. Números Enteros. 10
5.2. Números en Coma Flotante. 11
5.3. Números Complejos. 12
6. Secuencias. 12
6.1. Listas. 13
6.2. Tuplas 17
6.3. Rangos 19
6.4. Selección de subsecuencias basadas en índices (slices) 21
6.5. Operaciones predefinidas para secuencias. 23
7. Secuencias de caracteres. 24
7.1. Caracteres a números y viceversa. 26
7.2. Búsqueda de caracteres y conteo. 26
7.3. Operaciones relacionadas con el tamaña de letra. 27
7.4. Operaciones de identificación de cadenas. 28
7.5. Operaciones relacionadas con la codificación. 30
7.6. Traducciones, reemplazos de cadenas. 31
7.7. Manipulación de cadenas. 32
7.8. Justificado y alineación de cadenas de caracteres. 34
7.9. Formatear cadenas de caracteres. 35
7.10. Función para imprimir caracteres. 38
8. Conjuntos (SET y FROZENSET) 38
8.1. Funciones de actualización de conjuntos. 39
8.2. Funciones para operar conjuntos. 41
8.3. Operaciones condicionales para conjuntos. 42
9. Mapas (Diccionarios) 44
9.1. Explorar valores de los diccionarios. 44

9.2. Actualizar valores en diccionarios. 47
10. Iterables e Iteradores....49
10.1. Operadores para trabajar con iterables. 50
11. Expresiones Generadoras 54
11.1. Inicializar tipos por comprensión. 55
Listas por comprensión 56
Conjuntos por comprensión 56
Diccionarios por comprensión. 57
11.2. Inicializar objetos con expresiones generadoras. 58

1. Introducción a los tipos de datos

En Python, los tipos de datos se pueden construir y hacer a medida con el objetivo de modelar cualquier objeto de la vida real. Sin embargo, veremos los tipos de datos presentes en el núcleo de Python (built-in o predefinidos).

GRUPO	NOMBRE	TIPO	MUTABLE	EJEMPLOS
Numéricos	Entero	int	No	34, -12
	Punto Flotante	float	No	2.69, 6.8e9
	Complejos	complex	No	5j, 2+6j
Secuencias	Listas	list	Si	[1,2,3] [1, True, 'c', 5]
	Tuplas	tuple	No	(3,4,False)
	Secuencias numéricas	range	No	range(9)
Secuencias de texto	Cadenas de caracteres	str	No	'casa', "hombre", '''tecla''', """gato"""
Secuencias binarias	Cadenas binarias	bytes	No	b'coche'
	Cadenas binarias mutables	bytearray	Si	bytearray(b'Hola')
Conjuntos	Conjunto	set	Si	set([3,True,2]), {4, False, 12}
	Conjunto estático	frozenset	No	frozenset([2,'hola', True, 3])
Mapas	Diccionarios	dict	No	{'x':1, 'y':2}, dict(x=90, y=20)

Como se puede observar, Python provee de un gran número de tipos de datos fácilmente instanciables e intuitivos. Además, es lo suficientemente versátil para permitir que cualquier programador pueda definir sus propios tipos. La columna **mutable** hace referencia a una característica que veremos posteriormente.

La función **type** permite conocer el tipo de cualquier objeto den Python:

```
>>> type(45)
<class 'int'>
>>> type('libro')
<class 'str'>
>>> type({'x':1,'nombre':'antonio'})
<class 'dict'>
```

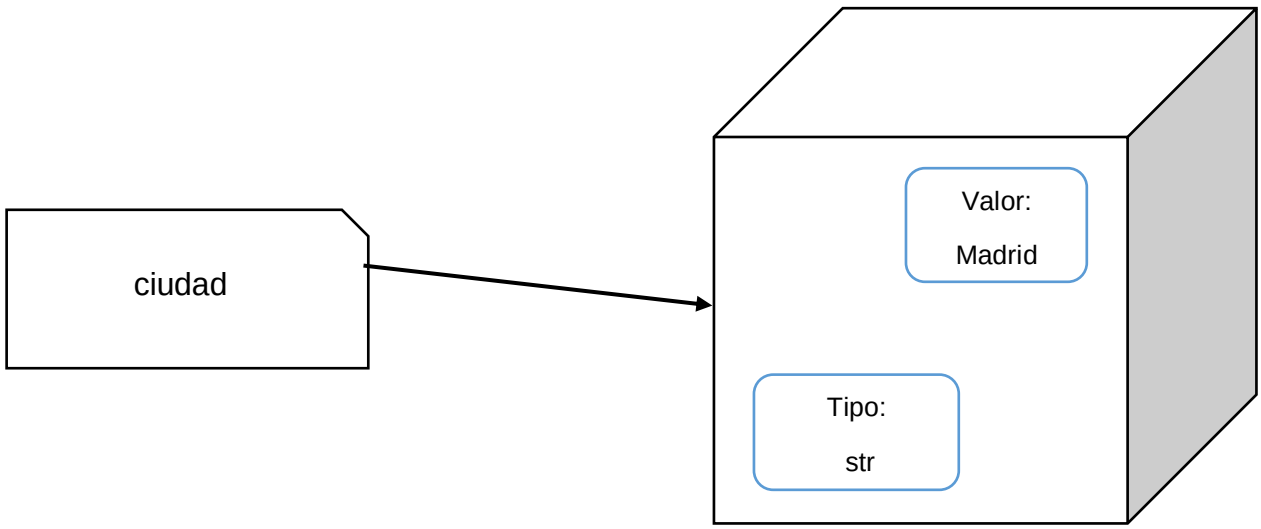
Si usamos la función **dir** igualmente en el terminal, se pueden ver todas las funciones disponibles para es tipo de dato:

```
>>> dir(4)
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc
__', '__eq__', '__float__', '__floor__', '__floordiv__', '__format__', '__ge__', '__getattribute__', '__getnewargs__'
, '__gt__', '__hash__', '__index__', '__init__', '__init_subclass__', '__int__', '__invert__', '__le__', '__lshift__'
, '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__', '__radd__', '__rand
__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__'
, '__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__', '__rtruediv__', '__rxor__', '__setattr
__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__', '__xor__', 'as_integer_ratio
', 'bit_length', 'conjugate', 'denominator', 'from_bytes', 'imag', 'numerator', 'real', 'to_bytes']
```

Como se puede observar, si utilizamos la función dir(4) obtenemos todas las operaciones disponibles para el tipo de dato **int.**

2. Literales, variables y datos en Python.

En Python, todos los tipos de datos son objetos y es el propio intérprete el que se encarga de gestionar la memoria y el acceso a ellos. Conceptualmente, estos objetos se pueden ver como dos componentes, por un lado **un puntero,** que guarda la dirección de la mm. en la que está alojado el dato y por otro lado, **la caja,** que representa el almacenamiento del objeto guardado en sí.



Existen dos formas de especificar el valor de los datos, mediante literales y mediante variables.

Los **literales** son el resultado de las expresiones o las propias formas primitivas de cada dato que ocupan espacio en memoria. Así tenemos como literales, por ejemplo, las cadenas de caracteres ‘Madrid’ o “Python”, o los números 1, 25, 158, 10.8, o cualquier tipo de dato que se defina desde su forma literal.

Las **variables** son uno de los componentes clave para cualquier lenguaje de programación. Permiten nombrar valores almacenados en memoria y manipular esos valores.

```
>>> ciudad='Madrid'
>>> país = "España"
>>> ciudad
'Madrid'
>>> país
'España'
```

En el ejemplo anterior se ha asignado a los literales ‘Madrid’ y “España” los nombres de variables **ciudad** y **país.**

Para nombrar variables en Python se usa la convención de **snake_case** que indica que los nombres deben estar en forma minúscula (variables locales) o mayúsculas (constantes) y unidos por “_”. Pueden contener números pero “siempre deben empezar por letra o “_”

Nombres válidos:

casa, _coche, numero1, PUNTO, valorReal, resultado_final

Nombres NO válidos:

1num, perro!, &resultado, python-3, valor Real.

Existen una serie de cadenas reservadas del lenguaje que no pueden ser usadas como nombres de variables:

```
>>> help('keywords')

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               break               for                 not
None                class               from                or
True                continue            global              pass
__peg_parser__      def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
```

Como hemos visto, para **inicializar** el valor de una variable se utiliza el concepto de **asignación,** que consiste en utilizar el "**=**". Esto permite asignar el contenido a la derecha de la ecuación al nombre de la izquierda. Las siguientes son formas válidas de asignación:

balance= 100

gasto = 50

beneficio = balance – gasto

num_ruedas = num_neumaticos= 4 (número de ruedas y número de neumáticos valen lo mismo)

estanterias , libros = 1, 5 (el 1 se asigna a estanterias y el 5 se asigna a libros)

La gestión de la memoria es otro de los puntos clave en cualquier lenguaje de programación, en especial en los de alto nivel. No se va a entrar a describir cómo gestiona Python la memoria pero diremos que cuenta con lo que se conoce como **recolector de memoria**. Este recolector, que se encuentra implementado en la máquina virtual, se encarga de liberar de forma automática el espacio de memoria que no se necesita. Gracias a esta herramienta, lo desarrolladores de Python no se tendrán que preocupar de la gestión de memoria como sí ocurre con otros lenguajes como C o C++.

Como se ha mencionado anteriormente, la característica de **mutabilidad** que se presenta en los tipos de datos y que hemos relacionado en la tabla de tipos, define si un dato puede ser mutado (cambiado) tras ser inicializado o si siempre mantiene el mismo valor, tanto para la variable original como para las demás referencias que tenga el valor. Pero no lo confundamos con variable o constante… Lo vemos y lo entendemos mejor con ejemplos:

```
>>> x, y = 1, 4
>>> z = x
>>> print(x,y,z)
1 4 1
```

Si ahora modificamos el valor de la variable z…

```
>>> z = 7
```

…y mostramos los valores igual que antes, observamos que **x** mantiene su valor original a pesar de que estaba igualado a **z**

```
>>> print(x,y,z)
1 4 7
```

Esto es así porque, como se define en la tabla de los tipos de datos, el tipo **int** es **NO MUTABLE**. Es decir, en este caso, **x** no cambia **z.**

¿Y qué ocurre si utilizamos un tipo de dato mutable?, veámoslo con el tipo de dato **List**

```
>>> lista=[12,5,9]
>>> lista_copia = lista
>>> lista_copia[1] = 6
>>> print(lista_copia)
[12, 6, 9]
>>> print(lista)
[12, 6, 9]
```

En el ejemplo anterior hemos creado una **lista** y le hemos asignado los valores **12,5,9**

Hemos creado una nueva lista a la que hemos denominado **lista_copia** que es una copia exacta de la lista inicial **lista.** Como puedes comprobar, si modificamos la copia de la lista, los cambios se producen igualmente sobre la lista original. Esto es a causa de que el tipo de dato **'list'** es **MUTABLE.**

3. Tipo Booleano.

El concepto de verdadero y falso en Python se moldea con dos valores: **True** y **False** (con la primera letra en mayúscula).

El constructor de objetos tipo booleano es **bool** y permite convertir cualquier objeto en su valor. Así tenemos:

```
>>> bool(1)
True
>>> bool(0)
False
```

Se puede decir que todos los objetos en Python se consideran verdaderos salvo los siguientes:

- Las constantes **False** y **None**
- Los valores numéricos interpretados por cero: 0, 0.0, 0j
- Objetos vacíos; ‘’, “”, (), dict(), set(), range(0), []

Los números booleano están construidos como enteros de manera que **True** tiene el valor **1** y **False** tiene el valor **0.**

3.1. Operaciones con booleanos.

Las operaciones lógicas son tres: **and, or** y **not.**

El operador **or** devuelve verdadero siempre y cuando alguno de sus operandos sea verdadero.

```
>>> True or False
True
>>> False or True
True
```

El operador **and** devuelve verdadero si y sólo si todos los operandos son verdaderos.

```
>>> True and False and True
False
>>> True and True and True and False
False
```

3.2. Cortocircuito lógico.

Esta propiedad se refiere al hecho de que Python no necesita evaluar las expresiones completas, sino que, si durante la evaluación de una expresión **and** se encuentra un elemento con valor “falso”, interrumpe la evaluación (cortocircuita) y devuelve ese valor.

Si observamos el ejemplo siguiente:

```
>>> 1 and True and 0 and x
0
>>> x
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'x' is not defined
```

…en el momento que se valora la condición **0,** se para la ejecución y se devuelve ese valor **0.** Si no fuese así y como podemos comprobar en la siguiente línea donde vemos que **x** es una variable que no está definida, nos devolvería el error Traceback…

4. Comparaciones.

En Python existen 8 tipos de comparadores básicos:

Operador	Ejemplo	Descripción
>	x > y	x es mayor que y
>=	x >= y	x es mayor o igual que y
<	x < y	x es menor que y
<=	x <= y	x es menor o igual que y
==	x == y	x es igual a y
!=	x != y	x es distinto de y
is	x is y	x es un objeto idéntico a y
is not	x is not y	x no es un objeto idéntico a y

Para los tipos de datos **iterables** (se verán más adelante) existen los operadores **in** y **not in** que serán muy útiles para saber si un objeto se encuentra o no en el tipo de dato.

```
>>> 23 in [12, 15, 23, 65]
True
>>> 'z' not in dict(x=1, y=36, z=25)
False
>>> 7 in range(10)
True
```

5. Tipos numéricos

En Python existen tres tipos numéricos que permiten expresar los **números enteros**, los **números en coma flotante** (reales) y los **números complejos** de forma sencilla y operar con ellos.

Las operaciones numéricas básicas son las siguientes:

Operador	Ejemplo	Descripción	Operación Reflexiva	Descripción
+	x + y	Suma de x e y	x += 5	Suma 5 a x
-	x – y	Resta de x e y	x -= 9	Resta 9 a x
*	x * y	Producto de x e y	x *= 3	Multiplica x por 3
/	x / y	División x entre y	x /=2	Divide x entre 2
//	x // y	División entera x entre y	x //=3	Divide de forma entera x entre 3
%	x % y	Módulo de x con y	x %=4	Aplica el módulo 4 a x
**, pow	x ** y, pow(x,y)	X elevado a y	x **=5	Eleva x a 5
abs	abs(x)	Valor absoluto de x		



Prueba a realizar distintas operaciones utilizando los distintos operadores.

5.1. Números Enteros.

Son los valores numéricos más simples e intuitivos. Se definen con el tipo **int** y la función constructora comparte el mismo nombre y representa números tanto negativos como positivos más el 0.

Ejemplos:

```
>>> 25 + 8
33
>>> -3-6
-9
>>> 4 + 15 / 6
6.5
>>> 4 + 15 // 6
6
>>> int(4 + 15 / 6)
6
>>> 2 ** 3
8
>>> pow(2,3)
8
>>> int(6.9)
6
```

En Python 3, los enteros pueden ser tan grandes como sea necesario, dado que **no tienen un número máximo fijo asignado.**

Como se ha visto con anterioridad, mediante el comando **dir** podemos conocer las operaciones disponibles de cualquier objeto.

```
>>> dir(int())
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__',
 '__doc__', '__eq__', '__float__', '__floor__', '__floordiv__', '__format__', '__ge__', '__getattribute__', '_
_getnewargs__', '__gt__', '__hash__', '__index__', '__init__', '__init_subclass__', '__int__', '__invert__', '
__le__', '__lshift__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__or__', '__pos__', '_
_pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '_
_rlshift__', '__rmod__', '__rmul__', '__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub_
_', '__rtruediv__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__true
div__', '__trunc__', '__xor__', 'as_integer_ratio', 'bit_length', 'conjugate', 'denominator', 'from_bytes', 'i
mag', 'numerator', 'real', 'to_bytes']
```

5.2. Números en Coma Flotante.

Los números en coma flotante o reales permiten no sólo la representación de números con decimales sino también la representación en notación científica de forma fácil.

El constructor y el tipo de esta clase se define con **float.**

```
>>> 9 / 2
4.5
>>> 3.14 * 2
6.28
>>> float(" 16.24")
16.24
>>> 8e+5
800000.0
```

A diferencia de los **int** los números en coma flotante tienen una precisión máxima. Esto se puede comprobar ejecutando la función **sys.float_info** que nos da esa información y alguna más:

```
>>> import sys
>>> sys.float_info
sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308, min=2.2250738585072014e-308,
 min_exp=-1021, min_10_exp=-307, dig=15, mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1
)
```

De igual forma, podemos obtener las operaciones disponibles para los números complejos:

```
>>> dir(float())
['__abs__', '__add__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__', '_
_float__', '__floor__', '__floordiv__', '__format__', '__ge__', '__getattribute__', '__getformat__', '__getnewargs__', '__g
t__', '__hash__', '__init__', '__init_subclass__', '__int__', '__le__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__'
, '__new__', '__pos__', '__pow__', '__radd__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '
__rmod__', '__rmul__', '__round__', '__rpow__', '__rsub__', '__rtruediv__', '__setattr__', '__setformat__', '__sizeof__', '
__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__', 'as_integer_ratio', 'conjugate', 'fromhex', 'hex', 'im
ag', 'is_integer', 'real']
```

5.3. Números Complejos.

Son una extensión de los **números reales** que permiten resolver operaciones que no son posibles con los números reales, como por ejemplo resolver la ecuación cuya solución sería x = i, siendo i un número imaginario. Esto números complejos tienen una parte real y una parte imaginaria. En Python están implementados igual que el resto de números con la diferencia de que para la parte imaginaria se añade una 'j' o 'J' al lado del número en lugar de una 'i' como es costumbre.

```
>>> 2 + 3j + 4
(6+3j)
```

El constructor es **complex.** Se pueden realizar operaciones básicas tanto entre números complejos como si es una combinación de complejos con enteros o reales.

```
>>> complex("1+4j")
(1+4j)
```

```
>>> dir(complex())
['__abs__', '__add__', '__bool__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '
__floordiv__', '__format__', '__ge__', '__getattribute__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subc
lass__', '__int__', '__le__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__pos__', '__pow__', '__radd
__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '__rmod__', '__rmul__', '__rpow__', '__rsub
__', '__rtruediv__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__truediv__', 'conjugate', 'im
ag', 'real']
```

6. Secuencias.

Los tipos de datos basados en secuencias se caracterizan por ser una concatenación de elementos, del mismo tipo o distintos, dispuestos de forma secuencial y que permiten realizar operaciones de forma sencilla e intuitiva. Para que un objeto se considere secuencia en Python, debe implementar el **protocolo de secuencia**. Para ello, la clase del objeto debe implementar el método mágico **__getitem__**, basado en números enteros comenzando desde el 0, que permite que todos los elementos puedan ser accedidos utilizando el índice de la posición a la que se quiere acceder con la función **index** y el método mágico **__len__**, el cual permite saber la longitud de la secuencia. Esto permitirá usar operaciones propias de secuencias como **len** o **count.**

Veamos los tipos basados en secuencias como son:

Listas, Tuplas y Rangos.

6.1. Listas.

Son secuencias de elementos de cualquier tipo y sin límite de longitud. El constructor y el tipo de dato es **list,** aunque existe una representación más usual: una secuencia de elementos separados por comas y rodeados por corchetes.

list ([1,2,3])

[1,2,3]

```
>>> a = list([1,2,3])
>>> b = [1,2,3]
>>> a
[1, 2, 3]
>>> b
[1, 2, 3]
>>> type(a)
<class 'list'>
>>> type(b)
<class 'list'>
```

Como se puede comprobar, hemos creado dos listas (a y b) de distinta forma.

A continuación presentamos las **diferentes operaciones** disponibles para manipulación de **listas** en Python:

Vamos a partir de la siguiente lista:

```
>>> lista=[1,2,3,4,5,6]
>>> lista
[1, 2, 3, 4, 5, 6]
```

- **lst.append(elem)**: añade el elemento **elem** al final de la lista **lst**

```
>>> lista.append(8)
>>> lista
[1, 2, 3, 4, 5, 6, 8]
```

- **lst.clear():** elimina todos los elementos de la lista **lst**

```
>>> lista.clear()
>>> lista
[]
```

- **lst.copy():** devuelve una copia de **lst.**

```
>>> nuevaLista=lista.copy()
>>> nuevaLista
[1, 2, 3, 4, 5, 6, 8]
```

- **lst.count(elem):** devuelve el número de veces que el elemento aparece en la lista.

```
>>> lista.count(4)
1
>>> lista.append(4)
>>> lista.count(4)
2
```

- **lst.extend(iter):** extiende la lista **lst** añadiendo todos los elementos del iterable **iter**

  De momento nos basta con saber que los objetos iterables son objetos capaces de ser recorridos iterando sobre los objetos que contienen. Así tenemos que son objetos iterables las propias listas, tuplas, rangos o las cadenas de caracteres.

```
>>> lista
[1, 2, 3, 4, 5, 6, 8]
>>> otraLista=[10,11]
>>> lista.extend(otraLista)
>>> lista
[1, 2, 3, 4, 5, 6, 8, 10, 11]
```

- **lst.index(elem[inicio[,final]]):** devuelve el índice de la posición que ocupa **elem** dentro de la lista **lst**. Los corchetes de inicio y final significan que son opcionales. Si se especifican determinarán el lugar donde buscar el elemento, aunque el índice siempre es relativo al inicio de la lista. El índice de una lista comienza en 0.

```
>>> lista
[1, 2, 3, 4, 5, 6, 8, 10, 11]
>>> lista.index(5)
4
```

En el ejemplo anterior, podemos comprobar que el elemento “5” se encuentra en la posición (índice) 4 comenzado por el 0 desde la izquierda.

```
>>> lista.index(7)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: 7 is not in list
```

Si buscamos un elemento que no se encuentra en la lista, nos devolverá una excepción del tipo **ValueError** indicando que el elemento no se encuentra en la lista.

- **lst.insert(pos,elem):** inserta el elemento **elem** en la posición anterior indicada por **pos.** Por ejemplo, si quisiéramos inserta un nuevo elemento en la primera posición de nuestra lista:

```
>>> lista
[1, 2, 3, 4, 5]
>>> lista.insert(0,6)
>>> lista
[6, 1, 2, 3, 4, 5]
```

- **lst.pop([pos]):** devuelve y elimina al mismo tiempo el elemento de la posición **pos.** Este parámetro es opcional y si se omite, se devuelve y elimina el **último** elemento de la lista.

```
>>> lista.pop()
5
>>> lista
[6, 1, 2, 3, 4]
```

```
>>> lista.pop(0)
6
>>> lista
[1, 2, 3, 4]
```

- **lst.remove(elem):** elimina la primera ocurrencia de **elem** en la lista. Si no encuentra **elem,** devuelve una excepción del tipo ValueError.

```
>>> lista
[1, 2, 3, 4, 4, 3]
>>> lista.remove(3)
>>> lista
[1, 2, 4, 4, 3]
```

- **lst.reverse():** invierte el orden de los elementos de la lista.

```
>>> lista
[1, 2, 4, 4, 3]
>>> lista.reverse()
>>> lista
[3, 4, 4, 2, 1]
```

- **lst.sort(key=None, reverse=False):** ordena las lista **lst.** Los parámetros son opcionales. **reverse=True** indica que se ordenará de mayor a menor. En el parámetro **key** se puede añadir cualquier función que sirva para ordenar, normalmente un tipo de función anónima denominada **lambda** (veremos posteriormente)

```
>>> lista=[5,8,6,25,1,14]
>>> lista
[5, 8, 6, 25, 1, 14]
>>> lista.sort()
>>> lista
[1, 5, 6, 8, 14, 25]
>>> lista.sort(reverse=True)
>>> lista
[25, 14, 8, 6, 5, 1]
```

Veamos ahora qué ocurre si tenemos una lista de números como la anterior y añadimos cadenas de caracteres (nombres). Si intentamos ordenarla vemos que nos devuelve un error…

```
>>> lista
[25, 14, 8, 6, 5, 1, 'Mario', 'Antonio', 'antonio']
>>> lista.sort()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: '<' not supported between instances of 'str' and 'int'
```

Como podemos comprobar, no soporta la ordenación de elementos de cadena y enteros.

Para solucionar esto podemos hacer uso de la función lambda de la siguiente manera:

```
>>> lista.sort(key=lambda x: str(x))
>>> lista
[1, 14, 25, 5, 6, 8, 'Antonio', 'Mario', 'antonio']
```

¿Qué hemos conseguido?. Lo que ha hecho la función lambda es definir cada uno de los elementos de la lista como **str** , es decir, como cadenas de caracteres y a partir de ahí, ordena todos los elementos como iguales.

6.2. Tuplas

Una tupla es un tipo de secuencia similar a una lista pero inmutable, por lo que una vez que se inicializa no se puede cambiar ninguno de sus elementos sin generar un nuevo objeto. Por este motivo, no se puede añadir, eliminar, ordenar…como ocurre con las listas.

El tipo de las tuplas coincide con el constructor **tuple.**

```
>>> tupla=tuple([1,2,3,4,5])
>>> tupla
(1, 2, 3, 4, 5)
```

De igual modo, cuando se añaden valores separados por coma en cualquier parte del código Python, estos se interpretan con una tupla de **N** elementos, igual que si se deja una coma al final de la sentencia:

```
>>> tupla2=5,6,9,8
>>> tupla2
(5, 6, 9, 8)
>>> type(tupla2)
<class 'tuple'>
```

```
>>> tupla3=25,
>>> tupla3
(25,)
>>> type(tupla3)
<class 'tuple'>
```

Las funciones asociadas a las tuplas son más reducidas en comparación con las listas. Más allá de las funciones propias de las secuencias de Python, se tienen las funciones **count** e **index.**

- **tpl.count(elem):** devuelve el número de veces que **elem** aparece en la tupla.

```
>>> tupla=1,2,5,6,5,4,2,5,6,5
>>> tupla
(1, 2, 5, 6, 5, 4, 2, 5, 6, 5)
>>> tupla.count(5)
4
```

- **tpl.index(elem[, inicio [, final]]):** devuelve el índice de la posición que ocupa el elemento **elem** dentro de la tupla. El índice comienza por 0. Si se especifican los valores opcionales inicio y final, se indica por dónde se debe comenzar y finalizar la búsqueda, aunque el índice devuelto siempre es relativo al inicio de la tupla. Si no se encuentra el elemento en la tupla, se devuelve una excepción del tipo ValueError.

```
>>> tupla.index(4)
5
>>> tupla.index(5)
2
```

```
>>> tupla.index(8)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: tuple.index(x): x not in tuple
```

6.3. Rangos

Los **rangos** son un tipo particular de secuencia de números enteros inmutable predefinidas en Python. Se crean utilizando el constructor **range,** que también determina el tipo de dato.

range(final): el parámetro final es un número entero que determina hasta dónde se deben generar números, comenzando desde el 0 y sumando 1 a cada paso:

```
>>> miRango=range(6)
>>> miRango
range(0, 6)
>>> list(miRango)
[0, 1, 2, 3, 4, 5]
```

Al asignar a la variable **miRango** el valor de **range(6)** estamos creando un rango con los valores que van de 0,6 sin incluir el 6, es decir, 0,1,2,3,4,5 (6 valores)

En el ejemplo anterior utilizamos el tipo Lista (list) para ver los valores del rango.

range(inicio, final, [paso]): otra forma de declarar un rango de valores. El parámetro **inicio** define el comienzo del rango. El valor **final** define el mayor valor del rango excluido el mismo. El parámetro opcional **paso** define el valor para añadir a cada iteración. Si se omite, el valor por defecto es **1.**

```
>>> miRango=range(5,15,2)
>>> list(miRango)
[5, 7, 9, 11, 13]
```

Cabe destacar que los índices pueden ser negativos:

```
>>> miRango=range(-10,10)
>>> list(miRango)
[-10, -9, -8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Las operaciones soportadas por los rangos son las siguientes:

- **rng.count(num):** devuelve el número de ocurrencias de num en el rango. Por norma, sólo se tiene una ocurrencia o ninguna de cada número entero.

```
>>> miRango.count(6)
1
>>> miRango.count(15)
0
```

- **rng.index(num):** devuelve el índice donde se encuentra el número **num** dentro del rango y si no se encuentra, devuelve una excepción del tipo ValueError.

```
>>> miRango.index(-3)
7
```

- **rng.start ; rng.step ; rng.stop :** son atributos que permiten conocer el número en el que comienza el rango, el paso numérico usado y el número final en el que parará el rango respectivamente.

```
>>> inicio, final, paso = 5, 15, 2
>>> miRango=range(inicio, final, paso)
>>> list(miRango)
[5, 7, 9, 11, 13]
>>> miRango.start, miRango.stop, miRango.step
(5, 15, 2)
```

Cabe destacar que las comparaciones de rangos se hacen comparando los valores propios del rango y no por los valores con los que se hayan definido. Así podemos tener rangos iguales cuando en apariencia son distintos. Por ejemplo:

```
>>> rango1=range(5,15,4)
>>> rango2=range(5,14,4)
>>> rango1==rango2
True
```

Son iguales porque si listamos sus valores:

```
>>> list(rango1)
[5, 9, 13]
>>> list(rango2)
[5, 9, 13]
```

6.4. Selección de subsecuencias basadas en índices (slices)

Una peculiaridad de los objetos que implementan el protocolo de secuencia es que pueden ser seleccionados por partes utilizando los índices a modo de subsecuencias, de manera que si tenemos una lista, tupla, rango o cualquier tipo de secuencia de **N** elementos, se pueden elegir desde **0** hasta **N** elementos y definir tanto el orden como el paso.

Así tenemos:

Partimos de la lista=[1,2,3,4,5,6,7,8]

sec[ind] → selección de un único elemento marcado por la posición de **ind.**

```
>>> lista[3]
4
```

sec[-ind]→ selecciona el elemento de la posición **ind** comenzando desde la última posición de la secuencia. Hay que tener en cuenta que la cuenta desde el fin de la secuencia es -1 para la última, -2 para la penúltima…y así sucesivamente.

```
>>> lista
[1, 2, 3, 4, 5, 6, 7, 8]
>>> lista[-1]
8
>>> lista[-2]
7
```

sec[ind_ini : ind_fin]→ selecciona una "subsecuencia" (grupo de elementos de la secuencia) desde la posición **ind_ini** hasta **ind_fin,** excluyendo la última posición.

```
>>> lista
[1, 2, 3, 4, 5, 6, 7, 8]
>>> lista[2:6]
[3, 4, 5, 6]
```

sec[ind_ini :]→ selecciona la "subsecuencia" desde la posición **ind_ini** hasta el final de la secuencia.

```
>>> lista
[1, 2, 3, 4, 5, 6, 7, 8]
>>> lista[2:]
[3, 4, 5, 6, 7, 8]
```

sec[ind_ini : ind_fin : paso]→ selecciona la "subsecuencia" desde la posición **ind_ini** hasta la posición **ind_fin** pero se define un **paso** para ir sumando en cada iteración.

Tomemos una lista más grande que definimos a partir de un rango:

```
>>> miLista=list(range(0,50,3))
>>> miLista
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48]
```

Ahora vamos a seleccionar una secuencia a partir de la lista anterior sumando en cada iteración 3.

```
>>> miLista[4 : 10 : 3]
[12, 21]
```



Reflexiona en el ejemplo anterior. ¿Por qué crees que no se ha seleccionado la "subsecuencia" [12, 21, 30] ?

sec[:ind_fin]→ selección de la secuencia desde el inicio hasta el final. Es similar a sec[0:] e incluso a sec[0:ind_final]

```
>>> miLista[0:17]
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48]
>>> miLista[0:]
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48]
>>> miLista[:17]
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48]
```

sec[:ind_fin:paso]→ selecciona la secuencia hasta **ind_fin** pero sumando el **paso** en cada iteración.

```
>>> miLista[:10:2]
[0, 6, 12, 18, 24]
```

sec[::paso]→ selecciona toda la secuencia pero utilizando el **paso** para recorrerlo.

```
>>> miLista[::5]
[0, 15, 30, 45]
```

sec[::] → devuelve una copia de la secuencia.

Cabe destacar que todos los **pasos** pueden ser negativos en cuyo caso actúan de la misma forma pero recorriendo la secuencia en orden inverso.

```
>>> miLista[::-1]
[48, 45, 42, 39, 36, 33, 30, 27, 24, 21, 18, 15, 12, 9, 6, 3, 0]
```

6.5. Operaciones predefinidas para secuencias.

Las secuencias, como **iterables que son**, tienen operaciones especiales que ayudan a trabajar con ellas de forma fácil y sencilla:

len(sec)→ devuelve el número de elementos de **sec.**

```
>>> lista
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> len(lista)
10
```

max(sec)→ devuelve el elemento máximo de **sec.**

```
>>> max(lista)
9
```

min(sec)→ devuelve el elemento mínimo de **sec.**

```
>>> min(lista)
0
```

in→ devuelve **True** el elemento por el que preguntamos está en la secuencia y **False** en caso contrario.

```
>>> 5 in lista
True
>>> 15 in lista
False
```

not in→ devuelve **True** si el elemento no está en la secuencia y **False** en caso contrario.

```
>>> 5 not in lista
False
>>> 15 not in lista
True
```

7. Secuencias de caracteres.

Las secuencias de caracteres son un tipo peculiar de las secuencias. Son conocidas como **cadenas de caracteres** o, simplemente, **cadenas.**

La unidad mínima de una cadena de caracteres es **un carácter** perteneciente a la tabla **Unicode.** Las cadenas de caracteres son secuencias **inmutables.**

En Python, los caracteres y las secuencias de caracteres son del tipo **str,** aunque suelen crearse como literales de cadena, rodeando un carácter, palabra o frase con comillas simples o dobles en bloques de una o de tres comillas, dependiendo de si se quiere tener una simple frase o un párrafo con saltos de línea. Veamos ejemplos:

```
>>> caracter='c'
>>> otroCaracter="b"
>>> cadena='casa'
>>> otraCadena="coche"
>>> cadenaLarga='esto es una frase'
>>> otraCadLarga="Esto es otra frase"
```

```
>>> caracter
'c'
>>> otroCaracter
'b'
>>> cadena
'casa'
>>> otraCadena
'coche'
>>> cadenaLarga
'esto es una frase'
>>> otraCadLarga
'Esto es otra frase'
```

Al existir dos tipos de comillas se pueden crear frases en las que se use un tipo u otro. Si se pretende usar el carácter de comilla doble, se rodea la frase con comillas simples y no es necesario utilizar el carácter de escape \' o \"

```
>>> cadena='esto es una frase con comillas simples que utiliza "comillas dobles" sin tecla de escape'
>>> cadena
'esto es una frase con comillas simples que utiliza "comillas dobles" sin tecla de escape'
```

Si se requiere crear frases con más de una línea (párrafos), se pueden usar comillas simples con el carácter de salto de línea de Python (\n) o se puede utilizar un bloque con tres comillas (dobles o simples) que permite escribir el párrafo de manera simple.

```
>>> parrafo='''Esto es un párrafo en dos líneas. Esta es la primera línea
... y esta es la segunda línea'''
>>> print(parrafo)
Esto es un párrafo en dos líneas. Esta es la primera línea
y esta es la segunda línea
```

Dado que hay diferentes formas de representar las cadenas, las **cadenas vacías** se pueden crear usando cada método. La cadena vacía es evaluada como **False** en comparaciones.

```
>>> '','','''''',""""""
('', '', '', '')
>>> bool('')
False
```

El constructor de los objetos de cadenas de caracteres es **str().** Por defecto devuelve una cadena vacía pero se le puede pasar por parámetro cualquier tipo de dato e intentará convertirlo en una cadena de caracteres.

```
>>> str()
''
>>> a=25
>>> str(a)
'25'
```

```
>>> import datetime
>>> str(datetime.datetime(2020,10,8,5,30))
'2020-10-08 05:30:00'
```

En el ejemplo anterior, importamos la librería **datetime** para poder ulilizarla como tipo de dato que pasamos al constructor **str.** Lo que se hace realmente es convertir los datos pasados al método **datetime** en un tipo fecha/hora. A continuación, el constructor **str** convierte ese dato fecha/hora en una cadena de caracteres.

Vamos a probar con un tipo de dato **bytes.** Para ello, lo primer que haremos será convertir una cadena de texto en tipo bytes con el método **encode.**

```
>>> 'Cigüeña'.encode()
b'Cig\xc3\xbce\xc3\xb1a'
```

Ahora vemos el paso contrario con el constructor **str():**

```
>>> str(b'Cig\xc3\xbce\xc3\xb1a',encoding='utf-8')
'Cigüeña'
```

Como puedes comprobar, al pasar el tipo de dato bytes o bytearray, es necesario que pasemos el parámetro en el que queremos codificar (encoding), en este caso, **utf-8.**

7.1. Caracteres a números y viceversa.

ord(char)→ devuelve el **número** Unicode del carácter pasado como parámetro.

```
>>> ord('a')
97
>>> ord('A')
65
```

chr(ind)→ devuelve el carácter en Unicode encontrado en la posición **ind** de la tabla Unicode.

```
>>> chr(100)
'd'
>>> chr(68)
'D'
```

7.2. Búsqueda de caracteres y conteo.

cad.count(sub[,ind_ini [,ind_fin]])→ devuelve el número de ocurrencias de la **sub** en **cad.** Los valores opcionales **ind_ini** e **ind_fin** determinan la secuencia dentro de **cad** donde se aplica la función.

```
>>> cadena='Esto es una cadena de texto'
>>> cadena.count('to')
2
>>> cadena.count('to',0,14)
1
```

Observa cómo en el ejemplo anterior hemos tratado las dos posibilidades. Contar cuántas ocurrencias de la subcadena “to” existen en la **cadena** de texto y en el segundo ejemplo, contar las ocurrencias de la misma subcadena pero limitando la búsqueda entre “Esto es una cadena” dado que el índice inicial es 0 y el final 14.

cad.find(sub[,ind_ini [,ind_fin]])→ devuelve el índice mínimo donde se encuentra la cadena **sub**. Si se pasan por parámetro los valores **ind_ini** e **ind_fin** la búsqueda se limita a la secuencia entre esos índices dentro de **sub.**

El resultado de la búsqueda si no se encuentra **sub** es **-1.**

```
>>> cadena
'Esto es una cadena de texto'
>>> cadena.find('cadena')
12
```

```
>>> cadena.find('cadena',15)
-1
```

Al especificar sólo el parámetro **ind_ini,** la secuencia llega hasta el final de la cadena.

cad.rfind(sub[, ind_ini [,ind_fin]])→ es igual que la anterior (find) con la diferencia de que lo que devuelve es el índice máximo.

```
>>> cadena.find('to')
2
>>> cadena.rfind('to')
25
```

cad.index(sub[, ind_ini [,ind_fin]])→ similar a **find** salvo que si no encuentra la subcadena **sub** devuelve una excepción del tipo ValueError.

cad.rindex(sub[, ind_ini [,ind_fin]])→ similar a rfind salvo que si no encuentra la subcadena **sub** devuelve una excepción del tipo ValueError.

7.3. Operaciones relacionadas con el tamaño de letra.

Ya sabemos que Python diferencia entre mayúsculas y minúsculas. A continuación presentamos las distintas operaciones para modificar el tamaño de las letras en una cadena.

cad.lower()→ devuelve la cadena **cad** con todos los caracteres en minúsculas.

```
>>> cadena='HOLA'
>>> cadena.lower()
'hola'
```

cad.upper()→ devuelve la cadena **cad** con todos los caracteres en mayúsculas.

```
>>> cadena='hola'
>>> cadena.upper()
'HOLA'
```

cad.title()→ devuelve la cadena **cad** donde cada palabra contenida en la cadena tiene el primer carácter en mayúscula.

```
>>> cadena='Esto es un título de una película'
>>> cadena.title()
'Esto Es Un Título De Una Película'
```

cad.capitalize()→ devuelve la cadena **cad** con la primera letra de en mayúscula y el resto en minúsculas.

```
>>> cadena='esto es el título de una película'
>>> cadena.capitalize()
'Esto es el título de una película'
```

cad.swapcase()→ devuelve la cadena **cad** cambiando cada carácter de mayúscula a minúscula y viceversa.

```
>>> cadena='Esto es un Título'
>>> cadena.swapcase()
'eSTO ES UN tÍTULO'
```

cad.casefold()→ devuelve la cadena **cad** de manera que se puede comparar con otra cadena sin importar las mayúsculas y/o minúsculas.

```
>>> cadena
'Esto es un Título'
>>> cadena2
'eSTO ES UN tÍTULO'
>>> cadena.casefold() == cadena2.casefold()
True
```

7.4. Operaciones de identificación de cadenas.

Python implementa por defecto funciones que sirven para evaluar si una cadena se parece a algún tipo de datos, como por ejemplo el número 1 con el carácter '1'…

cad.isdecimal()→ devuelve True si todos los caracteres de la cadena **cad** son decimales (0-9). En caso contrario devuelve False.

cad.isdigit()→ devuelve True si todos los caracteres de la cadena son dígitos o decimales. En caso contrario devuelve False.

cad.isnumeric()→ devuelve True si todos los caracteres de la cadena son numéricos, dígitos o decimales**.** En caso contrario devuelve False.

Como podemos comprobar, estas tres funciones son muy similares y la mayor diferencia se encuentra en el rango de caracteres que cubre cada una de ellas, siendo más limitada de menor a mayor rango isdecimal, isdigit e isnumeric en ese orden.

Seguramente nosotros nos servirá con utilizar la función **isdecimal()** para comprobar si tenemos números o letras.

```
>>> cadena='Hola'
>>> numerica='125'
>>> cadena.isdecimal()
False
>>> numerica.isdecimal()
True
```

cad.isascii()→ devuelve True si es una cadena vacía o si todos los caracteres pertenecen a la tabla ASCII. En caso contrario devuelve False.

```
>>> numerica.isascii()
True
```

cad.isalpha()→ devuelve True si todos los caracteres son alfabéticos y al menos hay un carácter. En caso contrario devuelve False.

```
>>> cadena='Este es el número 1'
>>> cadena.isalpha()
False
```

cad.isalum()→ devuelve True si todos los caracteres son alfanuméricos y al menos hay un carácter. En caso contrario devuelve False.

```
>>> cadena='1235'
>>> cadena.isalnum()
True
```

cad.isspace()→ devuelve True si todos los caracteres son espacios y al menos hay un carácter. En caso contrario devuelve False.

```
>>> cadena='    '
>>> cadena.isspace()
True
```

cad.islower()→ devuelve True si todos los caracteres de la cadena están en minúscula y al menos hay un carácter. En caso contrario devuelve False.

```
>>> cadena='hola mundo'
>>> cadena.islower()
True
```

cad.isupper()→ devuelve True si todos los caracteres de la cadena están en mayúscula y al menos hay un carácter. En caso contrario devuelve False.

```
>>> cadena='HOLA MUNDO'
>>> cadena.isupper()
True
```

cad.istitle()→ devuelve True si la cadena está en formato título. En caso contrario devuelve False.

```
>>> cadena='Esto Es Un Titulo'
>>> cadena.istitle()
True
```

7.5. Operaciones relacionadas con la codificación.

Todas las cadenas de caracteres en Python son de tipo Unicode, pero cuando se quiere guardar una versión de una cadena de caracteres como datos binarios o se intenta cambiar la codificación de la misma, es necesario instanciar desde Unicode una codificación específica. Esto se realiza utilizando la función **encode.**

cad.encode(encoding="utf-8", errors="strict")→ devuelve una versión de la cadena como objeto **bytes.** Por defecto, la codificación utilizada es utf-8 pero se puede utilizar cualquiera de las disponibles. El segundo parámetro se utiliza para manejar errores al intentar aplicar la codificación elegida.

- **strict**: valor por defecto que intenta hacer la codificación elegida y si no se puede devuelve una excepción ValueError
- **ignore**: se ignora cualquier error producido.
- **replace**: reemplaza el carácter mal formado por '?'.
- **backslashreplace**: reemplaza el carácter mal formado por '\'.
- **xmlcharrefreplace**: reemplaza el carácter mal formado por la referencia apropiada en XML

- **namereplace**: reemplaza el carácter mal formado por su representación en texto plano en Unicode usando \N

```
>>> cadena='El árbol'
>>> cadena.encode('utf-8')
b'El \xc3\xa1rbol'
```

Mira el siguiente ejemplo:

```
>>> cadena.encode('ascii')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
UnicodeEncodeError: 'ascii' codec can't encode character '\xe1' in position 3: ordinal not in range(128)
```

Como puedes comprobar, si intentamos llamar a la función con la codificación **ascii**, nos devuelve error dado que ascii no recoge el carácter 'á'. Para poder recoger y tratar el error, utilizamos la opción 'replace' de la siguiente forma:

```
>>> cadena.encode('ascii','replace')
b'El ?rbol'
```

Como puedes observar, no devuelve error sino que reemplaza el carácter mal formado por '?'.

7.6. Traducciones, reemplazos de cadenas.

Python permite construir cadenas a partir de otras, cambiando, carácter a carácter, elementos de una para generar otra nueva.

Las funciones encargadas de hacer este tipo de operaciones son:

cad.removeprefix(prefix→ devuelve una copia de la cadena original eliminando la cadena **prefix** si se encuentra al comienzo de **cad.** De lo contrario devuelve la cadena original.

```
>>> cadena="patata"
>>> cadena.removeprefix('pa')
'tata'
```

```
>>> cadena.removeprefix('ta')
'patata'
```

cad.removesufix(sufix)→ devuelve una copia de la cadena original eliminando la cadena **sufix** si se encuentra al final de **cad.** De lo contrario devuelve la cadena original.

```
>>> cadena.removesuffix('ta')
'pata'
```

```
>>> cadena.removesuffix('pa')
'patata'
```

cad.replace(old, new [, count])→ devuelve una copia de la cadena en la que se han sustituido todas las ocurrencias de la cadena en el parámetro **old** por la cadena del parámetro **new.** Si se facilita el valor **count** sólo se reemplazará el número de veces indicado.

```
>>> cadena.replace('a','e')
'petete'
```

```
>>> cadena.replace('a','e',2)
'peteta'
```

7.7. Manipulación de cadenas.

Python permite manipular las cadenas limpiando caracteres como sufijos o prefijos, separar cadenas o unir diferentes secuencias. Se hace a través de las siguientes funciones:

cad.strip([chars])→ devuelve una copia de la cadena con los caracteres **chars** eliminados. Si se omite la opción **chars** o se usa **None,** se eliminan los espacios en blanco.

```
>>> cadena='123abc123'
>>> cadena.strip('123')
'abc'
```

cad.rstrip([chars])→ devuelve una copia de la cadena con los caracteres **chars** eliminados pero sólo los que están al final de la cadena. No se trata del sufijo sino que las combinaciones de los mismos se eliminarán de la cadena. Si se omite la opción **chars** o se usa **None,** se eliminan los espacios en blanco.

```
>>> cadena='123abc123'
>>> cadena.rstrip('123')
'123abc'
```

cad.lstrip([chars])→ devuelve una copia de la cadena con los caracteres **chars** eliminados pero sólo los que están al comienzo de la cadena. No se trata del prefijo sino que las combinaciones de los mismos se eliminarán de la cadena. Si se omite la opción **chars** o se usa **None**, se eliminan los espacios en blanco.

```
>>> cadena='123abc123'
>>> cadena.lstrip('123')
'abc123'
```

cad.split(sep=None, maxsplit=-1)→ devuelve una lista de cadenas de caracteres. Usa el separador **sep** de la cadena original para delimitar cada cadena resultante. Si no se

especifica **sep** toma como separador el espacio en blanco; **maxsplit** sirve para limitar el número máximo de separaciones que se permiten. Si no se especifica se intentarán hacer todas las posibles.

```
>>> lista='Esto es una cadena'.split()
>>> lista
['Esto', 'es', 'una', 'cadena']
>>> lista[0]
'Esto'
```

Para ver mejor este ejemplo, se han guardado las cadenas en una lista. Ahora podemos acceder a cada cadena mediante el índice de la lista. De esa manera comprobamos que en la posición [0] de la lista tenemos la cadena 'Esto' que es resultado de la separación de la cadena inicial.

Veamos un ejemplo pasando el parámetros **maxplit.**

```
>>> cadena='uno,dos,tres,cuatro,cinco'
>>> cadena.split(',',3)
['uno', 'dos', 'tres', 'cuatro,cinco']
```

Como podemos comprobar, si utilizamos la función con la cadena del ejemplo, utilizamos como separador ',' y establecemos el parámetro **maxplit=3,** lo que hace es dividir la cadena original en 3 cadenas utilizando la separación de la ','.

cad.rsplit(sep=None, maxplit=-1)→ en este caso, la división de las cadenas comienza desde la derecha. Lo demás es exactamente igual.

```
>>> cadena='uno dos tres cuatro cinco'
>>> cadena.rsplit()
['uno', 'dos', 'tres', 'cuatro', 'cinco']
```

Como podemos comprobar, realmente así no se aprecia ningún efecto distinto con respecto a **split()**

Pero si ejecutamos el siguiente ejemplo, veremos que sí hay diferencia dado que la división comienza por el lado derecho, de manera que la cuenta de maxplit comienza también por el lado derecho.

```
>>> cadena.split(' ',3)
['uno', 'dos', 'tres', 'cuatro cinco']
>>> cadena.rsplit(' ',3)
['uno dos', 'tres', 'cuatro', 'cinco']
```

cad.splitlines(keepends=False)→ devuelve una lista de líneas de la cadena de caracteres original, separando las líneas usando los separadores de líneas (\n \r). Los separadores de línea no se añaden a las listas resultantes a no ser que se especifique con keepends=True.

```
>>> cadena='Esto es una cadena con varias líneas \n y estamos haciendo el ejemplo de separar \n líneas según el separador salto de línea'
>>> cadena.splitlines()
['Esto es una cadena con varias líneas ', ' y estamos haciendo el ejemplo de separar ', ' líneas según el separador salto de línea']
```

cad.partition(sep)→ divide la cadena original en la primera ocurrencia del separador **sep** y devuelve una **tupla** con tres elementos, la cadena antes del separador, el separador y la cadena a continuación del separador.

```
>>> cadena='Esto es un ejemplo de cadenas, separadas por la coma'
>>> cadena.partition(',')
('Esto es un ejemplo de cadenas', ',', ' separadas por la coma')
```

cad.rpartition(sep)→ idéntica a la función anterior con la diferencia de que la búsqueda de los separadores comienza a partir del final de la cadena original (derecha).

```
>>> cadena='Esto es un ejemplo de cadenas, separadas por la coma, y añadimos más texto para hacer la partición por la derecha'
>>> cadena.rpartition(',')
('Esto es un ejemplo de cadenas, separadas por la coma', ',', ' y añadimos más texto para hacer la partición por la derecha')
```

Observa que en el ejemplo anterior, la partición se realiza sobre la cadena desde la derecha hasta llegar a la primera ‘,’, dejando el resto de la cadena original.

cad.join(iter)→ devuelve una cadena de caracteres formada por la ‘unión’ de las cadenas del iterable **‘iter’.** Si alguno de los objetos del iterable no es una cadena de caracteres se producirá una excepción *TypeError.* El separador para unir las cadenas del iterador será la propia cadena a la que se le aplica esta función.

```
['1', '4', '6']
>>> '-'.join(lista)
'1-4-6'
```

7.8. Justificado y alineación de cadenas de caracteres.

cad.center(tam, relle=’ ‘) → devuelve una cadena de caracteres centrada, con una longitud especificada por el parámetro **tam** y completada con el carácter de relleno definido en el parámetro **relle.** Si no se especifica este parámetro se tomará el espacio en blanco. Si la cadena tiene un tamaño mayor que el propio parámetro **tam**, se devolverá tal y como es.

```
>>> cadena='Casa'
>>> cadena.center(20,'*')
'********Casa********'
```

cad.rjust(tam, relle=' ') → devuelve una cadena de caracteres alineada a la derecha con una longitud especificada por **tam** y completada con el carácter de relleno definido en el parámetro **relle**. Si no se especifica ese parámetro se tomará el espacio en blanco. Si la cadena tiene un tamaño mayor que el propio parámetro **tam**, se devolverá tal y como es.

```
>>> cadena.rjust(20,'*')
'****************Casa'
```

cad.ljust(tam, relle=' ') → devuelve una cadena de caracteres alineada a la izquierda con una longitud especificada por **tam** y completada con el carácter de relleno definido en el parámetro **relle**. Si no se especifica ese parámetro se tomará el espacio en blanco. Si la cadena tiene un tamaño mayor que el propio parámetro **tam**, se devolverá tal y como es.

```
>>> cadena.ljust(20,'*')
'Casa****************'
```

cad.zfill(tam) → devuelve una cadena de caracteres rellenada por la izquierda con caracteres '0' y de una longitud especificada por **tam**. Cuando se utilizan cadenas representando números el relleno se hace después de los signos '-' y '+'.

```
>>> '26'.zfill(10)
'0000000026'
>>> '-26'.zfill(10)
```

7.9. Formatear cadenas de caracteres.

Existe una función en Python que sirve para dar formato a una cadena de texto de forma flexible y legible.

cad.format(val1,val2) → devuelve una cadena copia de **cad** a la que se insertan valores en posiciones específicas usando espacios reservados.

La sintaxis más básica es:

cadena = "texto { } con { }".format(valor1, valor2)

Las llaves {} son marcadores de posición que se reemplazarán por los valores correspondientes pasados en el método format().

```
>>> cadena="Hoy es {} de {} de {}".format('11','noviembre','2024')
>>> cadena
'Hoy es 11 de noviembre de 2024'
```

Soporta muchos formatos. El ejemplo anterior es, quizás, el más simple pero podemos tener:

Indicando posiciones específicas.

```
>>> "El resultado del partido fué: Equipo Local: {0} - Equipo Visitante: {1}".format('3','1')
'El resultado del partido fué: Equipo Local: 3 - Equipo Visitante: 1'
```

Usando nombres en lugar de índices.

```
>>> "La capital de {region} es {ciudad}".format(ciudad="Mérida",region="Extremadura")
'La capital de Extremadura es Mérida'
```

Mediante formato de números decimales

```
>>> tipo=4.275
>>> "El tipo de interés del Banco es {:.2f}".format(tipo)
'El tipo de interés del Banco es 4.28'
```

Observa cómo ha redondeado el **tipo**.

Alineando texto.

```
>>> "Alineado a la izquierda: {:<10}.".format("texto")
'Alineado a la izquierda: texto     .'
```

```
>>> "Alineado a la derecha: {:>10}.".format("texto")
'Alineado a la derecha:      texto.'
```

```
>>> "Centrado: {:^10}.".format("texto")
'Centrado:   texto   .'
```

Con números.

```
>>> numero=4567.89
>>> "{:010.2f}".format(numero)
'0004567.89'
```

Como se puede observar, se rellena el número con ceros por la izquierda hasta completar las 10 posiciones definidas.

Otra forma, más antigua, de formatear cadenas de caracteres es hacer uso de **'%',** tanto en la plantilla como a la hora de pasar los argumentos que se quieren añadir. Con este formato se pueden pasar elementos a la cadena que se utiliza como plantilla de forma posicional o haciendo uso de un diccionario. Este método está en desuso, por lo que no se aconseja.

Con diccionario

```
>>> '%(nombre)s tuebe %(número)02d tipos de comillas'%{'nombre': 'Python', 'número': 2}
'Python tuebe 02 tipos de comillas'
```

Con paso de parámetros directamente.

```
>>> 'Hay %d %s por la carretera' % (4,'coches')
'Hay 4 coches por la carretera'
```

En Python existen otro tipo de cadenas que permiten formatear de forma clara y concisa un valor de tipo **str.** Son las cadenas **f-strings.** Para ello se añade el carácter '**f'** o **'F'** antes de la cadena para indicar que es una **f-string.**

Veamos un ejemplo usando el formateo con **'%**' y el mismo ejemplo con **f-string.**

```
>>> puntos=[1,2]
>>> vehiculo='coche'
>>> color='verde'
```

```
>>> 'El %s de color %s irá desde %d hasta %d' % (vehiculo, color, puntos[0], puntos[1])
'El coche de color verde irá desde 1 hasta 2'
```

```
>>> f'El {vehiculo} de color {color} irá desde {puntos[0]} hasta {puntos[1]}'
'El coche de color verde irá desde 1 hasta 2'
```

La ventaja de **f-strings** es que permiten hacer operaciones, llamar a funciones o acceder a variables.

Las **f-strings** cumplen con una formato en el que nosotros no vamos a profundizar pero que viene recogido en la documentación oficial de Python (si es de tú interés):

https://docs.python.org/3/reference/lexical_analysis.html#formatted-string-literals

Veamos algún ejemplo:

```
>>> número, entero = 23.412, 82
>>> nombre1, nombre2 = 'Pepe', 'Óscar'
>>> f'{número} {entero} {nombre1} {nombre2}'
'23.412 82 Pepe Óscar'
```

```
>>> f'Usando funciones: {chr(entero)}'
'Usando funciones: R'
```

```
>>> f'Usando expresiones: {nombre1 if número > entero else nombre2}'
'Usando expresiones: Óscar'
```

7.10. Función para imprimir caracteres.

La función por defecto para imprimir caracteres en Python es **print()**. Soporta un número indeterminado de cadenas separadas por ‘,’ y permite definir el separador y el carácter final de la cadena a imprimir. El separador por defecto es el espacio en blanco y el carácter de final es el salto de línea, pero se pueden modificar con los valores **sep** y **end**

```
>>> print('Hola. Esto es una cadena de presentación')
Hola. Esto es una cadena de presentación
```

```
>>> print("Modificamos","el separador y","el final",sep='*', end=';\n')
Modificamos*el separador y*el final;
```

8. Conjuntos (SET y FROZENSET)

Los conjuntos son una colección no ordenada de elementos únicos y que en Python se representan con **set** o **frozenset.**

Pueden contener cualquier tipo de dato y mezclarlo sin problema. Además se encargan de tener un, y sólo un, elemento igual. Todos sus elementos deben ser **hasheables,** lo que significa que nunca cambiarán durante su tiempo de vida.

La principal diferencia entre **set** y **frozenset** es que set es mutable y frozenset es inmutable y hasheable.

Los constructores de estos tipo de datos tienen el mismo nombre que sus tipos: **set** y **frozenset.**

Además, para la creación del tipo **set** se puede usar una versión simplificada que consta de un iterador rodeado de los caracteres ‘{‘ ‘}’.

```
>>> conjunto1=set()
>>> conjunto2=frozenset()
```

Se crean los conjuntos vacíos.

```
>>> conjunto1=set([1,2,3,4,5])
>>> conjunto2={1,2,3,4,5}
>>> conjunto1
{1, 2, 3, 4, 5}
>>> conjunto2
{1, 2, 3, 4, 5}
```

Se crean los conjunto con un argumento iterable.

```
>>> conjunto1=set([1,2,2,3,4])
>>> conjunto1
{1, 2, 3, 4}
```

Como puedes observar, si intentas crear un conjunto repitiendo alguno de los elementos, no lo guarda.

```
>>> conjunto1=set(range(2,20,2))
>>> conjunto1
{2, 4, 6, 8, 10, 12, 14, 16, 18}
```

Creamos un conjunto mediante un **rango**.

8.1. Funciones de actualización de conjuntos.

Estas funciones sólo se aplican a los conjuntos mutables **set**, dado que los no mutables, **frozenset,** no se pueden modificar.

```
>>> conjunto=set(range(0,20,2))
>>> conjunto
{0, 2, 4, 6, 8, 10, 12, 14, 16, 18}
```

conj.add(elem)→ añade el elemento **elem** al conjunto **conj.**

```
>>> conjunto.add(20)
>>> conjunto
{0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20}
```

conj.remove(elem)→ elimina el elemento **elem** del conjunto **conj.** Si no encuentra el elemento devuelve una excepción de tipo KeyError.

```
>>> conjunto.remove(20)
>>> conjunto
{0, 2, 4, 6, 8, 10, 12, 14, 16, 18}
```

conj.discard(elem)→ elimina **elem** del conjunto **conj.** Si no lo encuentra no devuelve ninguna excepción.

```
>>> conjunto.discard(20)
>>> conjunto
{0, 2, 4, 6, 8, 10, 12, 14, 16, 18}
```

conj.pop()→ devuelve y elimina un elemento arbitrario de **conj.**

```
>>> conjunto.pop()
0
>>> conjunto
{2, 4, 6, 8, 10, 12, 14, 16, 18}
```

conj.clear()→ elimina todos los elementos del conjunto **conj,** pero deja el conjunto vacío.

```
>>> conjunto.clear()
>>> conjunto
set()
```

conj.update(iter)→ añade los elementos del iterador **iter** al conjunto **conj.**

```
>>> conjunto.update(range(20,25))
>>> conjunto
{2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 21, 22, 23, 24}
```

conj.difference_update(iter)→ elimina los elementos del conjunto **conj** que encuentra en **iter.**

```
>>> conjunto.difference_update([20,21,22,23,24])
>>> conjunto
{2, 4, 6, 8, 10, 12, 14, 16, 18}
```

conj.intersection_update(iter)→ mantiene en el conjunto **conj** sólo los elementos que están al mismo tiempo en **conj** y en el iterador **iter.**

```
>>> conjunto.intersection_update([2,4,6,8])
>>> conjunto
{8, 2, 4, 6}
```

8.2. Funciones para operar conjuntos.

Estas operaciones están presentes tanto en conjunto mutables(set) como inmutables(frozenset)

```
>>> conjuntoA=set(['Ana','Maria','Marta'])
>>> conjuntoA
{'Ana', 'Marta', 'Maria'}
```

```
>>> conjuntoB=frozenset([25,52,34])
>>> conjuntoB
frozenset({25, 34, 52})
```

conj.union(iter)→ crea un nuevo conjunto mediante la unión de los elementos del conjunto **conj** y los elementos del iterador **iter** que puede ser otro conjunto. Se puede realizar la unión de la forma conjuntoA | conjuntoB… que es equivalente a **union.**

```
>>> conjuntoC=conjuntoA.union(conjuntoB)
>>> conjuntoC
{34, 52, 'Maria', 'Ana', 25, 'Marta'}
```

conj.intersection(iter)→ crea un nuevo conjunto con los elementos que están en el conjunto **conj** y al mismo tiempo en **iter,** que puede ser otro conjunto. Se puede realizar la intersección de la forma conjuntoA & conjuntoB…que es equivalente a **intersection.**

```
>>> conjuntoC
{34, 52, 'Maria', 'Ana', 25, 'Marta'}
>>>
>>>
>>> conjuntoD=set(['Maria',25,'Pablo','82'])
>>> conjuntoD
{'Maria', 25, '82', 'Pablo'}
>>> conjuntoE=conjuntoC.intersection(conjuntoD)
>>> conjuntoE
{'Maria', 25}
```

conj.difference(iter)→ crea un nuevo conjunto con los elementos de **conj** que no están en **iter.** Se puede realizar la diferencia de la forma conjuntoA – conjuntoB…que es equivalente a **difference.**

```
>>> conjuntoF=conjuntoD.difference(conjuntoE)
>>> conjuntoF
{'Pablo', '82'}
```

conj.symmetric_difference(conj_b)→ crea un nuevo conjunto con los elementos que están en **conj** o en **conj_b** per no en ambos al mismo tiempo. Se puede realizar la diferencia simétrica de la forma conjuntoA ^ conjuntoB…que es equivalente a **symmetric_difference.**

```
>>> conjuntoC
{34, 52, 'Maria', 'Ana', 25, 'Marta'}
>>> conjuntoD
{'Maria', 25, '82', 'Pablo'}
>>> conjuntoS=conjuntoC.symmetric_difference(conjuntoD)
>>> conjuntoS
{'Pablo', 34, 'Ana', 52, '82', 'Marta'}
```

conj.copy()→ crea un nuevo conjunto como copia exacta de **conj.**

```
>>> conjuntocopia=conjuntoS.copy()
>>> conjuntocopia
{'Pablo', 34, 52, '82', 'Ana', 'Marta'}
```

8.3. Operaciones condicionales para conjuntos.

Podemos comprobar si un conjunto pertenece a otro o si tienen elementos en común. De igual modo, al estar implementados en Python como iteradores, las operaciones se pueden usar para comprobar el tamaño y si un elemento pertenece a un conjunto o no.

```
>>> conjunto=set([1,2,3,4,5,6,7])
>>> conjunto
{1, 2, 3, 4, 5, 6, 7}
```

len(conj)→ devuelve un número natural (entero positivo) con el número de elementos del conjunto.

```
>>> len(conjunto)
7
```

elem in conj→ devuelve True si **elem** está en **conj.** En caso contrario devuelve False.

```
>>> 4 in conjunto
True
```

elem not in conj→ devuelve True si **elem** no está presente en **conj.** En caso contrario devuelve False

```
>>> 9 not in conjunto
True
```

conj.isdisjoint(conj_b)→ devuelve True si no existen elementos en común entre los dos conjunto, es decir, si la intersección entre ambos conjuntos es vacía.

```
>>> conjunto
{1, 2, 3, 4, 5, 6, 7}
>>> conjuntoB
{8, 9, 10, 11, 12}
>>> conjunto.isdisjoint(conjuntoB)
True
```

conj.issubset(conj_b)→ devuelve True si todos los elementos de **conj** están en **conj_b.**

```
>>> conjunto
{1, 2, 3, 4, 5, 6, 7}
>>> conjuntoC
{3, 4, 5}
>>> conjuntoC.issubset(conjunto)
True
```

conj.issuperset(conj_b)→ devuelve True si todos los elementos de **conj_b** están en **conj.**

```
>>> conjunto=set([1,2,3,4,5,6])
>>> conjuntoB=set([3,4,5])
>>> conjunto.issuperset(conjuntoB)
True
```

9. Mapas (Diccionarios)

Los mapas son estructuras contenedoras de datos usadas para guardar información en forma de pares de valores, formados por una clave única y un valor asociado a la misma.

En Python, los objetos que representan a los mapas son los diccionarios, que son objetos mutables que mapean objetos hasheables, a modo de claves, con un objeto arbitrario que representa el valor para cada clave.

Los mapas son del tipo ***dict*** y se pueden crear de cuatro formas diferentes:

```
>>> diccionarioA=dict(x=1, y=2)
>>> diccionarioA
{'x': 1, 'y': 2}
>>> diccionarioB=dict([('x',1),('y',2)])
>>> diccionarioC=dict(zip(['x','y'],[1,2]))
>>> diccionarioD={'x':1, 'y':2}
```

Podemos comprobar que todos los diccionarios son iguales:

```
>>> diccionarioA==diccionarioB==diccionarioC==diccionarioD
True
```

Otra forma de crear diccionarios es desde otro diccionario con la función **copy** o el método de clase **fromkeys,** que crea un nuevo diccionario desde un iterador en el que se especifican las claves y un valor opcional que se utilizará para inicializar el diccionario.

```
>>> d=dict.fromkeys(range(3),'nombre')
>>> d
{0: 'nombre', 1: 'nombre', 2: 'nombre'}
>>> dd=d.copy()
>>> dd
{0: 'nombre', 1: 'nombre', 2: 'nombre'}
>>> dd[0]='nuevo nombre'
>>> dd
{0: 'nuevo nombre', 1: 'nombre', 2: 'nombre'}
```

9.1. Explorar valores de los diccionarios.

Una vez creados los diccionarios se pueden comprobar las claves y los valores mediante distintas operaciones:

Las siguientes operaciones, .keys(), .values(), .items(), devuelven un objeto tipo **dict_view** que son vistas (views) sobre el contenido del diccionario, de manera que recogen las

modificaciones. Estos objetos dict_view pueden ser **dict_keys**, **dict_values** y **dict_items** según el método que se utilice.

dicc.keys()→ devuelve un objeto tipo dict_keys con las claves del diccionario.

dicc.values()→ devuelve un objeto tipo dict_values con los valores del diccionario.

dicc.items()→ devuelve un objeto tipo dict_items con los pares clave-valor del diccionario.

```
>>> diccionario={1:'Gato', 2:'Perro', 3:'Mono'}
>>> diccionario.keys()
dict_keys([1, 2, 3])
>>> diccionario.values()
dict_values(['Gato', 'Perro', 'Mono'])
>>> diccionario.items()
dict_items([(1, 'Gato'), (2, 'Perro'), (3, 'Mono')])
```

dicc.get(k [, valor_defecto])→ devuelve el valor asociado a la clave **k.** Si no existe la clave devuelve **None** o el valor por defecto si se ha especificado.

```
>>> diccionario.get(3)
'Mono'
>>>
>>> diccionario.get(6)
>>>
>>> diccionario.get(6,'Sin valor')
'Sin valor'
```

dicc[k]→ devuelve el valor asociado a la clave **k.** Si no lo encuentra devuelve una exception de tipo KeyError.

```
>>> diccionario[3]
'Mono'
```

```
>>> diccionario[6]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 6
```

list(dicc)→ devuelve una lista de las claves que contiene el diccionario **dicc.**

```
>>> list(diccionario)
[1, 2, 3]
```

len(dicc)→ devuelve el número de elementos que contiene el diccionario **dicc.**

```
>>> len(diccionario)
3
```

elem in dicc→ devuelve True si el elemento **elem** está en el diccionario **dicc** como clave**.**

elem not in dicc→ devuelve True si el elemento elem NO está en el diccionario **dicc** como clave.

```
>>> 1 in diccionario
True
>>> 5 in diccionario
False
>>> 1 not in diccionario
False
>>> 5 not in diccionario
True
```

iter(dicc)→ devuelve un objeto iterador sobre las claves de **dicc.** Es similar a realizar **iter(dicc.keys())**

Para ver mejor este ejemplo, lo haremos con un pequeñísimo programa en Python:

```
#declaramos el diccionario
diccionario={1:'Rojo',2:'Azul',3:'Marrón'}
#almacenamos el iterador
iterador=iter(diccionario)
#utilizamos un "for" para recorrer el iterador
for x in iterador:
  print(diccionario[x]) #sacamos por pantalla el valor para cada clave del diccionario
```

reversed(dicc)→ devuelve un objeto iterador sobre las claves de **dicc** pero en orden inverso al orden de inserción. Es similar a realizar **reversed(dicc.keys())**

Utilizamos el ejemplo anterior.

```
#declaramos el diccionario
diccionario={1:'Rojo',2:'Azul',3:'Marrón'}
#almacenamos el iterador
iterador=reversed(diccionario)
#utilizamos un "for" para recorrer el iterador
for x in iterador:
  print(diccionario[x]) #sacamos por pantalla el valor para cada clave del diccionario
```

9.2. Actualizar valores en diccionarios.

Los diccionarios son objetos **mutables**, por lo que se pueden modificar sin problemas tras su inicialización, añadiendo, eliminando o actualizando cualquier clave o valor de los mismos.

dicc[k] = elem → asigna el valor de **elem** a la clave **k**

```
>>> diccionario
{1: 'Gato', 2: 'Perro', 3: 'Mono'}
>>> diccionario[1]='Perro'
>>> diccionario
{1: 'Perro', 2: 'Perro', 3: 'Mono'}
```

dicc.update(opciones)→ permite actualizar una o varias claves y valores de un diccionario. Como parámetros tenemos varias opciones:

- dicc.update(dicc2)→ utiliza un segundo diccionario **dicc2** para actualizar el primero **dicc.** Todas las claves de dicc2 que no estén en dicc se añadirán con sus respectivos valores. Las que sí estén, se sobrescribirán.

```
>>> diccionario
{1: 'Perro', 2: 'Perro', 3: 'Mono'}
>>>
>>> diccionario2={1:'Gato',4:'Tigre'}
>>> diccionario.update(diccionario2)
>>> diccionario
{1: 'Gato', 2: 'Perro', 3: 'Mono', 4: 'Tigre'}
```

- dicc.update(iterador_pares)→ para actualizar un diccionario se puede usar un iterador de cualquier clase (lista, tupla, etc) que tenga pares de objetos en los que el primero sea hasheable (int, float, str, tuple…) dado que se utilizará como clave. El segundo elemento será usado como valor.

```
>>> diccionario
{1: 'Hiena', 2: 'Mono', 3: 'Serpiente', 4: 'Camello', 5: 'Caballo'}
>>> diccionario.update([(1,'Jaguar'),(5,'Cerdo')])
>>> diccionario
{1: 'Jaguar', 2: 'Mono', 3: 'Serpiente', 4: 'Camello', 5: 'Cerdo'}
```

del dicc[k]→ elimina la clave y su valor asociado del diccionario **dicc.**

```
>>> del diccionario[1]
>>> diccionario
{2: 'Mono', 3: 'Serpiente', 4: 'Camello', 5: 'Cerdo'}
```

dicc.pop(k [,valor_por_defecto])→ si **k** es una clave del diccionario **dicc**, devuelve el valor y elimina la clave, valor del diccionario. Si no existe la clave, se devuelve la excepción **KeyError** a no ser que se haya establecido un **valor_por_defecto.**

```
>>> diccionario.pop(2)
'Mono'
>>> diccionario
{3: 'Serpiente', 4: 'Camello', 5: 'Cerdo'}
>>> diccionario.pop(6,'No existe')
'No existe'
```

dicc.popitem()→ devuelve un par clave-valor en orden LIFO (último insertado, primero en salir) hasta vaciar el diccionario **dicc.** Si el diccionario está vacío devuelve la excepción **KeyError.**

```
>>> diccionario.popitem()
(5, 'Cerdo')
>>> diccionario.popitem()
(4, 'Camello')
>>> diccionario.popitem()
(3, 'Serpiente')
>>> diccionario.popitem()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'popitem(): dictionary is empty'
```

dicc.setdefault(k [,valor_por_defecto])→ si la clave **k** existe en el diccionario **dicc**, devuelve el valor de la clave. Si no está, crea una clave **k** con el valor **None** o **valor_por_defecto** si se ha especificado.

```
>>> diccionario.setdefault(1,'León')
'León'
>>> diccionario
{1: 'León'}
```

dicc.clear()→ elimina todos los elementos del diccionario **dicc.**

```
>>> diccionario
{2: 'Piraña', 3: 'Elefante', 4: 'Camello'}
>>> diccionario.clear()
>>> diccionario
{}
```

10. Iterables e Iteradores

Los objetos **iterables** en Python son objetos capaces de ser recorridos iterando sobre los elementos que contienen. Sólo necesitan implementar la función mágica __iter__.

Algunos ejemplos de objetos iterables que ya hemos visto son las listas, las tuplas o las cadenas de caracteres.

Los objetos **iteradores** son aquellos que cumplen con el protocolo de los iteradores e implementan las funciones mágicas __iter__ y __next__. La función __**next**__ devuelve el siguiente valor del objeto y una excepción del tipo **StopIteration** cuando no quedan elementos disponibles e el iterable.

Un iterador es un objeto que **sirve para recorrer** los elementos de un **iterable**, uno a uno, sin necesidad de acceder directamente a los índices.

Ejemplo:

En el siguiente código hemos creado una pequeña lista de números que será nuestro **iterable.** A continuación declaramos la variable iterador que será, como no puede ser de otra forma, nuestro **iterador.** Este nos permitirá recorrer la lista mediante la función **next** hasta que no queden elementos…

```
# Un iterable (una lista)
mi_lista = [10, 20, 30]

# Crear un iterador a partir del iterable
iterador = iter(mi_lista)

# Usar el iterador para recorrer los elementos uno a uno
print(next(iterador))  # Devuelve 10
print(next(iterador))  # Devuelve 20
print(next(iterador))  # Devuelve 30

#Si intentamos avanzar más allá del final, dará un error StopIteration
print(next(iterador))  # Esto genera una excepción.
```

El resultado mostrado por terminal será:

```
10
20
30
Traceback (most recent call last):
  File "f:\Mi Material 24_25\@vanza- Programación en Python\Ejemplos\pruebas.py", line 13, in <module>
    print(next(iterador))  # Esto genera una excepción.
StopIteration
```

10.1. Operadores para trabajar con iterables.

Python soporta múltiples funciones para trabajar con iterables e incluso convertirlos en iteradores.

min→ esta función devuelve el valor mínimo de los que se le pasen como parámetro. Se puede usar de las siguientes formas:

min(iterable [,key, valor_por_defecto])→ como parámetro obligatorio se pasa un objeto iterable y como opcional una función para comparar valores. Adicionalmente se puede pasar un valor por defecto en caso de que el iterador esté vacío y evitar que se devuelva la excepción **ValueError.**

min(arg1, arg2,…args[,key])→ puede recibir un número indefinido de argumentos con un mínimo de 2 y opcionalmente una función (key) que se usará para el orden de los elementos.

Ejemplo:

```
# Encuentra el número con el menor valor absoluto
numeros = [-10, -5, 1, 5, 10]
resultado = min(numeros, key=abs)
print(resultado)  # Salida: 0
```

Salida:

```
1
```

max→ esta función devuelve el valor máximo de los que se le pasen como parámetro. Se puede usar de las siguientes formas:

max(iterable [,key, valor_por_defecto])→ como parámetro obligatorio se pasa un objeto iterable y como opcional una función para comparar valores. Adicionalmente se puede pasar un valor por defecto en caso de que el iterador esté vacío y evitar que se devuelva la excepción **ValueError.**

max(arg1, arg2,…args[,key])→ puede recibir un número indefinido de argumentos con un mínimo de 2 y opcionalmente una función (key) que se usará para el orden de los elementos.

Ejemplo:

```
# Encuentra el número con el menor valor absoluto
numeros = [-20, -5, 1, 5, 10]
resultado = max(numeros, key=abs)
print(resultado)  # Salida: 0
```

Salida:

```
-20
```

iter→ esta función devuelve un objeto iterador, dependiendo de los parámetros usados tenemos:

iter(objeto)→ el parámetro debe ser un objeto tipo colección que soporte la iteración (es decir, que implemente la el método mágico __**iter**__) o el protocolo de secuencia (con el método mágico __**getitem**__ con argumentos enteros comenzando en 0) .Si no implementa alguno de estos dos protocolos, devuelve la excepción **TypeError.**

iter(objeto, centinela)→ cuando se pasan dos argumentos, el primero debe ser un objeto "callable" (que implemente el método mágico __call__ como puede ser cualquier función) y será llamado en cada iteración del bucle sin argumentos. Si en alguna de esas iteraciones el objeto callable devuelve el valor establecido como "centinela", se produce la excepción **StopIteration** y para.

```
#La siguiente instrucción, imprime una lista
#con los valores devueltos por el iterador
#que se aplica sobre la función pop
#hasta que se devuelve el valor centinela=3

print(list(iter([1,2,3,4,5].pop, 3)))
```

Recuerda que la función **pop** sobre una lista, devuelve el último valor de la lista y lo elimina, de manera que, a medida que el iterador va recorriendo la lista, llama a la función **pop**, devuelve el último elemento y lo elimina de la lista. Así hasta que encuentra el "centinela" 3, momento en el que aborta y devuelve la excepción comentada.

Salida:

```
[5, 4]
```

enumerate(iterable, start=0)→ devuelve un objeto **enumerate,** que es una tupla de dos elementos donde el primero es un entero que determina el índice (comenzando por **start**) y el segundo es el elemento en la posición del índice del iterable.

Ejemplo de un programa muy sencillo con un bucle **for** (veremos más adelante).

```
#declaramos una lista que servirá de iterable
nombres = ["Ana", "Luis", "Carlos", "Marta"]
#la propia función enumerate actúa de iterador sobre la lista,
#recorriéndola y comenzando con el valor de start
for indice, nombre in enumerate(nombres, start=1):
    print(f"{indice}: {nombre}")
```

Salida por terminal:

```
1: Ana
2: Luis
3: Carlos
4: Marta
```

A partir del ejemplo anterior, vamos a realizar un pequeño programa que nos permita crear un diccionario con los valores de la lista siguiente:

['Tigre','León','Elefante','Cebra']

De manera que las claves del diccionario comiencen en el índice 1.

El resultado debe ser: {1:'Tigre',2:'León',3:'Elefante',4:'Cebra']

```
#declaramos una lista que servirá de iterable
nombres = ['Tigre','León','Elefante','Cebra']
#Declaramos un diccionario vacio
dicc={}
#Utilizamos el la función enumerate domo iterador para recorrer
#las lista e ir introduciendo clave:valor en el diccionario
for indice, nombre in enumerate(nombres, start=5):
    dicc.update({indice:nombre})

#Sacamos por terminal el diccionario.
print(dicc)
```

Salida:

```
{5: 'Tigre', 6: 'León', 7: 'Elefante', 8: 'Cebra'}
```

any(iterable)→ devuelve True si alguno de los elementos del iterable es verdadero. Si el iterable está vacío devuelve False.

```
print(any([5<6,6<4,8>9]))
```

Como el elemento de la lista 5<6 es cierto, el resultado es True, independiente del resto de elementos.

all(iterable)→ sólo devuelve True si todos los elementos del iterable son verdaderos.

En el ejemplo anterior, devuelve False.

```
print(all([5<6,6<4,8>9]))
```

map(funcion, iterable,…)→ esta función devuelve un iterador que irá aplicando la función que se le pasa como primer parámetro a cada elemento de iterable y devolverá uno a uno los resultados. Se pueden añadir más iterables que pasarán sus elementos a la función de forma paralela, hasta que alguno de los iterables se quede sin elementos, de manera que el número total de elementos devueltos por la función **map** será la longitud del menor iterable de los pasados.

Ejemplo:

```
iterador=map(lambda x:x**2, range(5))
print(list(iterador))
```

Salida: `[0, 1, 4, 9, 16]`

En el ejemplo anterior, el parámetro **función** utilizamos “lambda x:x**2”. **lambda** es una función “anónima” de Python que se puede definir en una sola línea que es ideal para funciones sencillas o aquellas que son temporales, es decir, que no necesitas almacenar.

Su sintaxis es:

lambda **argumentos** : **expresión**

argumentos: son los parámetros que recibe la función que pueden ser 0 o más.

expresión: lo que la función devuelve que debe ser una única línea.

Ejemplo: la tabla de multiplicar del 3

```
y=3
multiplicax3=map(lambda x: x * y,range(1,11))
print(list(multiplicax3))
```

list([iterable])→ crea una lista con los valores obtenidos tras iterar un **iterable** hasta llegar al final del mismo. En los ejemplos anterior lo hemos utilizado.

reversed(secuencia)→ devuelve un iterador en orden inverso desde la secuencia que se le pasa como parámetro.

```
print(list(reversed([1,2,3,4,5,6])))
```

Salida: `[6, 5, 4, 3, 2, 1]`

zip(*iterables, strict=False)→ genera un iterador como agregación de cada elemento en la posición de cada iterable que se le pase como parámetro que como mínimo serán 2 iterables, devolviendo tuplas de tantos elementos como iterables se usen. La longitud final del iterador será igual a la menor longitud de todos los iterables pasados.

Ejemplo:

```
#observa que de los iterables pasados, uno tiene sólo tre elementos.
iterador=zip([1,2,3,4], [5,6,7,8],[9,10,11])
#eso hará que la salida devuelva tres tuplas
print(list(iterador))
```

Salida:

```
[(1, 5, 9), (2, 6, 10), (3, 7, 11)]
```

filter(funcion, iterable)→ genera un iterador con los elementos del iterable que al pasarlos como argumentos a la **funcion**, devuelven True. Si la **funcion** está definida como **None**, se devolverán sólo los elementos del iterable que sean True, excluyendo los False.

Ejemplo: Vamos a realizar de nuevo un ejemplo utilizando como función **lambda.**

```
iterador=filter(lambda x:x<500,[1,1024,501,6000,25])
print(list(iterador))
```

Como puedes observar, la función “lambda” evalúa que el parámetro recibido del iterador (en este caso una “lista”) sea menor de 500. La salida serán sólo aquellos valores que cumplen la comparación:

Salida: `[1, 25]`

Como habrás podido comprobar, los iteradores e iterables son muy útiles y muy utilizados en Python dado que multitud de sus objetos devuelven iteradores en diferentes formatos.

11. Expresiones Generadoras

Las expresiones generadoras son estructuras parecidas a las funciones que devuelven varios valores que se almacenan en un objeto **generador** que se comporta como un iterable.

Una característica importante a tener en cuenta es que los valores se almacenan en el generador de uno en uno, de manera que va evaluando de forma “perezosa” la generación de cada elemento en cada iteración usando la función **__next__.**

Principalmente se componen de un bucle **for** que va iterando sobre un **iterador.** Potencialmente puede contener más bucles **for** o una condición por un **if** para filtrar valores que no se quieran devolver del iterador.

Ejemplo:

```
generador = (x ** 2 for x in [1,2,3,4])

print(next(generador))
print(next(generador))
print(next(generador))
print(next(generador))
```

Salida:

```
1
4
9
16
```

Otro ejemplo con **if:**

```
generador = (x / 2 for x in [10,5,6,14,9] if x > 9)

print(list(generador))
```

Salida:

```
[5.0, 7.0]
```

Observa que en la salida, que hemos definido como una lista, nos devuelve los valores de la división de 10 y 14, omitiendo el resto de valores que no cumplen la condición de x > 9

Observa el siguiente ejemplo:

```
generador = (x / 2 for x in [10,5,6,14,9] if x > 9)

print(list(generador))

print(list(generador))
```

Salida:

```
[5.0, 7.0]
[]
```

Al igual que ocurre con los iteradores, cuando ha iterado sobre todos sus valores, ya no tienen "nada" que devolver. Si lo hiciésemos utilizando la función **next(),** obtendríamos la excepción **StopIteration**.

11.1. Inicializar tipos por comprensión.

Visto el concepto de expresiones generadoras, podemos comprender mejor que Python permite inicializar algunos objetos de forma compacta y elegante usando prácticamente la misma sintaxis que las expresiones generadoras. Es lo que se denomina **inicialización por comprensión.**

Consiste en una única expresión seguida de **al menos** una clausula **for** y **ninguna o más** clausulas **for** o **if.** Los elementos del nuevo contenedor serán los producidos por cada iteración de cada **for** desde la izquierda a la derecha y desde el bloque más interno al externo.

Los objetos que soportan la inicialización por comprensión son las listas, los diccionarios y los conjuntos.

Listas por comprensión

Idénticas a las listas convencionales pero la inicialización se lleva a cabo haciendo uso de la sintaxis de la comprensión rodeada de "[" y "]".

Ejemplo:

```
lista_c=[x * 2 for x in [2,3,4,5,]]
print(lista_c)
```

Salida:

```
[4, 6, 8, 10]
```

Otro ejemplo:

```
lista_c=[x + 5 if x > 4 else x - 10 for x in range(10)]
print(lista_c)
```

Salida:

```
[-10, -9, -8, -7, -6, 10, 11, 12, 13, 14]
```

Uno más:

```
ma = 'Casa'
mb='Ferro'
lista_c=[a + b for a,b in zip(ma,mb)]
print(lista_c)
```

Salida:

```
['CF', 'ae', 'sr', 'ar']
```

Es importante destacar que, aunque se utilice una sintaxis similar a las expresiones generadoras, el objeto resultante es una lista totalmente guardada en memoria, a diferencia de un generador.

Conjuntos por comprensión

Idénticos a los conjunto convencionales pero se inicializan con la sintaxis de comprensión rodeada de "{" y "}". Recuerda que al tratarse de conjuntos, sus elementos son únicos.

Ejemplo:

```
conjunto={x for x in 'Casa'}
print(conjunto)
```

Salida:

```
{'s', 'C', 'a'}
```

Otro ejemplo:

```
conjunto={x for x in range(1,20) if (x % 2)==0}
print(conjunto)
```

Salida:

```
{2, 4, 6, 8, 10, 12, 14, 16, 18}
```

Uno más:

```
ma = 'Casa'
mb='Ferro'
conjunto_c={a + b for a,b in zip(ma,mb)}
print(conjunto_c)
```

Salida:

```
{'CF', 'sr', 'ae', 'ar'}
```

Diccionarios por comprensión.

Idénticos a los diccionarios convencionales pero se inicializan con la sintaxis de comprensión. La sintaxis es simple y hace uso de pares separados por ":" para definir las claves y los valores. Se usa un mínimo de un bucle **for**, aunque se pueden usar más. También se pueden usar expresiones, tanto para filtrar valores de cada bucle como para expresar los valores o claves de la forma que se desee.

Ejemplo:

```
diccionario={i:v for i,v in enumerate(range(5))}
print(diccionario)
```

Salida:

```
{0: 0, 1: 1, 2: 2, 3: 3, 4: 4}
```

Otro ejemplo:

```
diccionario={x:x*2 for x in (range(1,5))}
print(diccionario)
```

Salida:

```
{1: 2, 2: 4, 3: 6, 4: 8}
```

Uno más:

```
nom=['Juan','Antonio','Pilar','Alberto']
notas=[8,9.3,6,3.5]
diccionario={nombre:nota for nombre,nota in zip(nom,notas)}
print(diccionario)
```

Salida:

```
{'Juan': 8, 'Antonio': 9.3, 'Pilar': 6, 'Alberto': 3.5}
```

11.2. Inicializar objetos con expresiones generadoras.

Aunque las inicializaciones por compresión sólo soportan tres tipos de datos, como se ha visto (listas, conjuntos y diccionarios), se pueden usar las expresiones generadoras para inicializar otros tipos de datos, aprovechando el carácter iterativo de las mismas.

Ejemplos:

```
cadena='-'.join(chr(x + 65) for x in range(10))
print(cadena)
```

Salida:

```
A-B-C-D-E-F-G-H-I-J
```

Otro ejemplo:

```
nombre=['Alberto','Marta']
numero=['1','2']
tupla=tuple(x+' ' +y for x in nombre for y in numero)
print(tupla)
```

Salida:

```
('Alberto 1', 'Alberto 2', 'Marta 1', 'Marta 2')
```